Mikkel, Hannibal og Abdallah - Hand-in Assigment 4

Consensus

Mikkel Vincent Hedegaard (mivh@itu.dk)
Hannibal Marcellus Munk (hmun@itu.dk)
Abdallah Jawher Alhamedi (aalh@itu.dk)

November 8, 2026 • Group : Mikkel, Hannibal og Abdallah • https://github.com/IntVincent0/consensus

---

1 A) - Describe how your system meets the System Requirements (R1, R2, and R3)

R1: Specification A node requests access to the critical section by calling `Enter()`. Each node does this at random intervals. Once access is granted, the node writes an entry with its id and Lamport timestamp to a shared log file, which is the critical section, and then calls `Exit()`. Since each node has a unique port number, each node is identified by their port number. We'll refer to the port number as a node's id.

R2: Safety To enter the critical section, a node sends a `Request` RPC (containing its Lamport timestamp and id) to every other node and must receive a reply from all of them. A node receiving a request replies immediately, unless it is in the critical section (HELD), or it also wants the critical section (WANTED) and its own request has priority. In those cases the reply is deferred until the node exits. Priority is given to the lower Lamport timestamp; ties are broken by the lower node id (port number). Since two nodes can never both have priority over each other, at most one node can collect all replies at a time, so at most one node is in the critical section.

R3: Liveliness Requests are totally ordered by (Lamport timestamp, id). Because of this ordering there can be no cycle of nodes waiting for each other, so the system cannot deadlock. When a node calls `Exit()`, it sets its state to RELEASED and sends all its deferred replies. Since Lamport clocks only increase, every waiting request eventually becomes the oldest one and is granted, so no node starves. This assumes that nodes do not crash and that every node in the critical section eventually call `Exit()`.

2 B) Provide a discussion of your algorithm, using examples from your logs (as per Technical Requirement 5)

As we can see in the logs below, a node will only enter the critical section once it has received a reply from the other two nodes in the system. This is logged as `[<port>] deferring reply to <port>`, which shows which nodes will receive a reply once the node's state changes to "released".

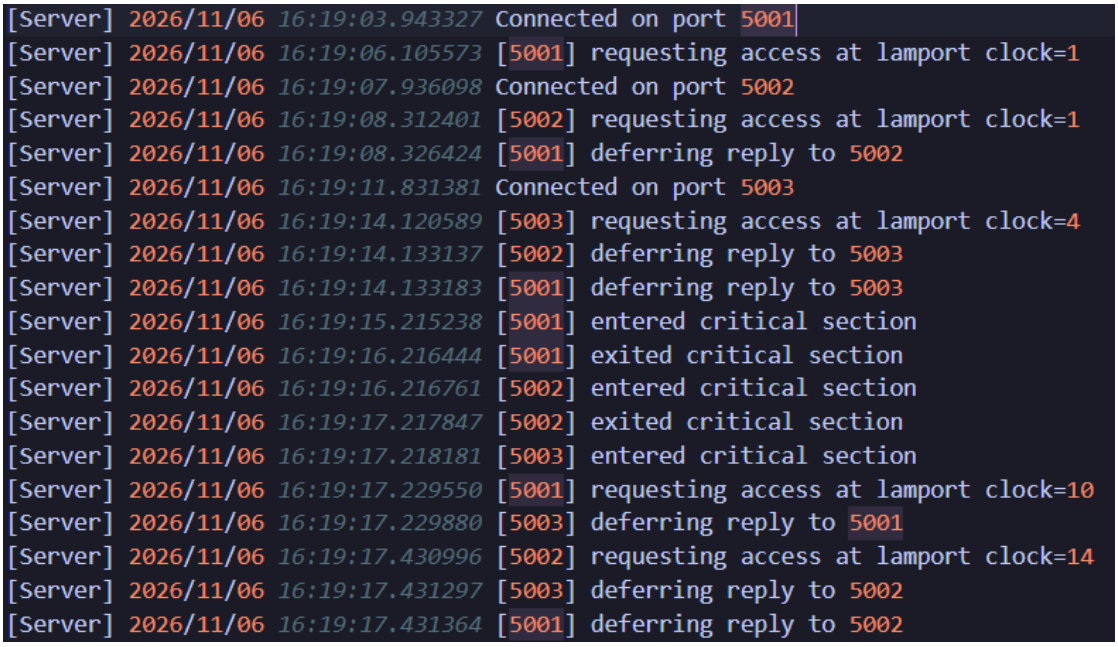


Figure 1: Snippet from node.log which is created when running node.go

In the example, the nodes on ports 5001, 5002 and 5003 request access to the critical section in that order. Nodes 5001 and 5002 both defer their reply to 5003. Once 5001 is done, it replies to 5002 and 5003, giving 5002 access (since it has now received replies from both 5003 and 5001). Once 5002 is done, 5003 receives its deferred reply and enters the critical section. This can be seen taking place from 16:19:14 to 16:19:17. It demonstrates R1 and R2, since each node gains access in the same order as it requested it, and no two nodes are ever in the critical section at the same time. Assuming that exit is eventually called, this also demonstrates R3. However, it also means that the entire system will deadlock if a node crashes.